

Methode = Unterprogramm, d.h.

parametrisierter Anweisungsblock mit Namen.

Er kann über seinen Namen aufgerufen werden.

Hinterher wird Code an aufrufenden Stelle fortgesetzt.

Ablauf beim Methodenaufruf

- Aktueller Parameter wird ausgewertet
(Bsp: Berechnung von $\text{betrag1} + \text{betrag2}$ ergibt 1570.22)
- Parameterübergabe: Werte der akt. Parameter werden in die formalen Parameter kopiert.
(Bsp: kapital wird auf 1570.22 gesetzt.)
- Ausführung des Methodenruffs.
- Beendigung der Methode bei "return".
Ergebnis wird an die aufrufende Stelle zurückgeliefert.
(Bsp: gewinn wird auf 1617.3266 gesetzt.)

2 Arten von Methoden:

- Prozeduren (Rückgabetyp void)
- Funktionen (Rückgabetyp $\neq$ void, return nötig)

Hier ist return möglich.

Sinn von Prozeduren:

- Ausgabe (z.B. drucke)

- Ausgabe (z.B. drucke)
- Seiteneffekte (z.B. Sortier-Methode) ← später

Parameterübergabe

verschiedene Arten in der Informatik

- | | |
|-------------------|---------------------------------------|
| call by value | } bei vielen imperativen Sprachen |
| call by reference | |
| call by name | } vor allem bei funktionalen Sprachen |

call by value

- akt. Parameter (im Methodenaufruf) wird ausgewertet
(Bsp: s ist schon ausgewertet, hat Wert 2.1)
- Wert des akt. Parameters wird in den formalen Parameter der Methode kopiert.
(Bsp: r bekommt den Wert von s)
- Änderung des formalen Parameters der Methode bewirkt keine Änderung des akt. Parameters.
(Bsp: s hat zum Schluss immer noch den Wert 2.1)

Speicherverwaltung bei Blöcken von Anweisungen

- Jedem Block entspricht ein Speicherbereich (frame) auf dem Laufzeitkeller (runtime stack), in dem die Werte der Variablen des Blocks gespeichert sind.
- Beim Eintritt in einen neuen Block wird der zugehörige Speicherbereich "oben" auf dem Laufzeitkeller


Zugehörige Speichersbereich oben auf dem Laufzeitkeller angelegt (allocate).

↑
im Sinne der Wachstumsrichtung.
Wir zeichnen Keller meist so,
dass sie nach unten wachsen.

- Der zuletzt angelegte Speichersbereich wird als erstes wieder gelöscht. (Last in - First out: Prinzip des Kellerspeichers)
- Variable ist zugreifbar in dem Block, in dem sie deklariert wurde, und in Unterblöcken.
- Java: Neue Variablen in inneren Blöcken dürfen nicht so heißen wie Var. in äußeren Blöcken.

```
int x;  
...  
{  
...  
double x; nicht erlaubt  
...  
}  
...
```

Laufzeitkeller bei Methoden

- gleiches Prinzip: Methodenrumpf ist auch ein Block
- Für jeden Methodenaufruf wird auf Laufzeitkeller ein neuer Speichersbereich angelegt.
- Beim Aufruf der Methode kommt der neue Speichersbereich oben auf den Keller.
- In Methoden dürfen Variablen genauso heißen wie an der aufrufenden Stelle.

- In Methoden dürfen Variablen genauso eingesetzt werden an der aufrufenden Stelle.
- Man kann aus der Methode nicht auf die Variablen der aufrufenden Stelle zugreifen.

Call by reference

- Aktueller Parameter ist Variable (z.B. s)
- Formaler Parameter der Methode wird Verweis / Referenz / Zeiger auf aktuellen Parameter (z.B. r zeigt auf s, r wird mit s identifiziert)
- Jede Änderung des formalen Parameters bewirkt Änderung des akt. Parameters.
(Bsp: s hätte zum Schluss den Wert 4.6).

Geht so in vielen Sprachen, aber nicht in Java.

Aber: Man kann eine Art call-by-reference in Java simulieren, bei Methoden, deren Parameter keinen primitiven Datentyp haben.

Da Objekte (und Arrays) als Referenzen gespeichert werden, bewirkt eine Änderung über den formalen Parameter der Methode eine Änderung, die auch über den aktuellen Parameter sichtbar ist.

Das ist ähnlich, aber nicht gleich zu call-by-reference. Wenn dem formalen Parameter ein neues Objekt zugewiesen wird, dann hat das keine Auswirkung auf

zugewiesen wird, dann hat das keine Auswirkung auf den aktuellen Parameter.

⇒ Java kopiert immer den Wert des aktuellen Parameters in den formalen Parameter.

Da nicht-primitive Datentypen über Referenzen gespeichert werden, entspricht das einer eingeschränkten Form von call-by-reference.

Informationen von der aufrufenden Stelle an die Methode sollten immer über Parameter übergeben werden, nicht über globale Variablen:

```
public static int x;
```

← globale Variable

```
public static void f(...) {  
    x = x + 1;  
}
```

```
public static void main() {  
    x;  
    f(...);  
    x;  
}
```

meist schlechter Stil:
x sollte lokale Var. von f sein und ihr Wert sollte f als Parameter übergeben werden.

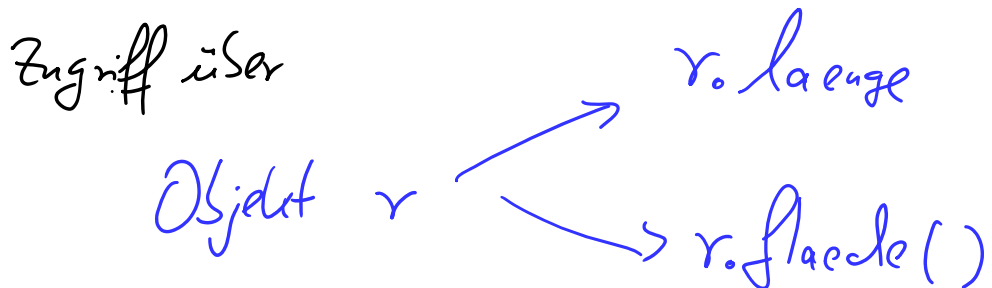
Bsp: für eine Prozedur mit gewünschtem Seiteneffekt:
"satiere"

Vararg-Parameter

- Vararg-Parameter sind Arrays.
- Man darf in dem aktuellen Parameter anstelle eines Arrays auch beliebig viele Array-Einträge (mit Kommas getrennt) schreiben.
- Vararg-Parameter nur als letzter formaler Parameter

Statische Methoden und Attribute

- nicht-statische Methoden/Attribute sind Eigenschaften des jeweiligen Objekts



Innerhalb einer nicht-statischen Methode kann man direkt auf nicht-statische Attribute/Methoden des aktuellen Objekts zugreifen.

- statische Methoden/Attribute sind Eigenschaften der ganzen Klasse, d.h. sie hängen nicht vom konkreten Objekt ab.

Bsp: Aufruf über Rechteck.flächenberechnung
↑
Klasse

↑
protokolliert mit,
wie oft eine
Flächenberechnung
stattfindet

Ausgabe: 0
1
2

Java erlaubt auch

r.flächenberechnung oder
s.flächenberechnung

Ist aber dasselbe wie

Rechteck.flächenberechnung

Wenn man Eigenschaften ohne aufruft:

- bei nicht-statischen Eigenschaften ist das aktuelle Objekt gemeint
(ist in statischen Methoden nicht möglich)
- bei statischen Eigenschaften ist die aktuelle Klasse gemeint

Aber: Wenn das Objekt explizit angegeben wird, dann ist der Zugriff auf seine nicht-statischen Eigenschaften auch in statischen Methoden möglich (d.h. `r.laeenge` oder `r.flaeche()` geht auch in statischen Methoden, aber `laenge` oder `flaeche()` nicht).

Generell: Alle Methoden, die eine Eigenschaft des Objekts ausdrücken (und auf nicht-statische Eigenschaften zugreifen) sollten nicht-statisch sein.

Weiteres Bsp für eine nicht-statische Methode
`toString()`

Diese Methode wird von `ID.print(ln)` aufgerufen.

⇒ So kann man `ID.print(ln)` auch für Objekte eigener Klassen benutzen.

Aufzählungstypen (Enumerations)

illustrieren die Verwendung von statischen Attributen

- spezielle Form der Klassendeklaration

- Spezielle Form der Klassendeklaration
- Man darf keine neuen Elemente von Enum-Klassen erzeugen:

`Tag s = new Tag ();` verboten

- Für jede Enum-Klasse werden bestimmte Methoden automatisch erzeugt.
- Enum-Klasse darf beliebige weitere Attribute und Methoden enthalten.
- switch Anweisung / Ausdruck ist nicht nur für Fallunterscheidungen anhand eines int-, char- oder String-Werts möglich, sondern auch für Enum-Werte.
- switch - Ausdruck muss immer eine vollständige Fallunterscheidung haben. Das geht bei Enums auch ohne default.

Gültigkeit von Bezeichnern

Welcher Bezeichner ist wo sichtbar?

Was passiert bei Bezeichnern mit gleichem Namen?

Grund:

- Modularität: Verwendung gleicher Namen in verschiedenen Prog.-Teilen ist kaum zu vermeiden.

zu vermeiden.

- Wiederverwendung gleicher Bezeichner kann die Lesbarkeit erhöhen.

Generelle Regeln:

- Deklaration eines Bezeichners gehört zum jeweiligen Block:

```
{  
    ...  
    int x; ← AS hier kann x benutzt werden.  
    ...  
}
```

← Hier ist x nicht mehr sichtbar.

- Variablen sind erst nach ihrer Deklaration sichtbar.
Bei Methoden und Klassen gilt das nicht.

```
void f(...) {  
    ...  
    g(...);  
    ...  
}  
  
void g(...) {  
    ...  
    f(...);  
    ...  
}
```

- Namensgleiche Bezeichner in einer Methode überdecken die Bezeichner außerhalb der Methode:

```
class A {
```

```
    int x;
```

```
    int y;
```

```
    void f(int y) {
```

```
        x
```

```
        double x;
```

```
    }
```

```
}
```

← Dieses y wird in der Methode f vom formalen Parameter y überdeckt

← Ab hier wird dieses x überdeckt.

- In einem Block müssen Bezeichner verschieden sein, es sei denn, sie bezeichnen verschiedenartige Elemente. (Variable, Methode, Klasse dürfen gleichen Namen haben.)

```
{
    int x;
    double x;
}
```

aber
erlaubt
wäre:

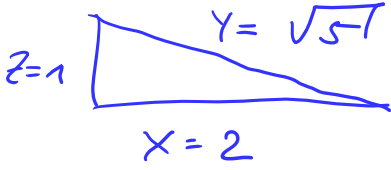
```
{
    int x;
    void x(...) {
    }
}
```

- Methoden mit verschiedenartiger Parameterliste dürfen gleichen Namen haben (⇒ überladene Methoden, später)

dürfen gleichen Namen haben ($\Rightarrow$ reservierte Wörter, später)

Bsp:

- Die statische Variable x aus Globalität wird vom formalen Parameter der `main`-Methode überschrieben.
- In `setze` werden die Attribute x, y, z von den formalen Parametern x, y, z überschrieben.
- Aber `d.x` ist das x -Attribut des Dreiecks `d`.

`d` wird das Dreieck 

- Beim Aufruf von `d.flaeche()`:
 - in `p = ...` bedeutet x : `d.x`
 - ab `y = ...` wird das Dreieck-Attribut y überschrieben
 - in `x * y / 2` ist x das Dreieck-Attribut
 y die neue Var. der Methode